IEEE Global Student Challenge
Federated Learning Backdoor Attack and Defense Challenge

Participant Guide

Contents

1 Introduction

Machine learning systems are increasingly trained using data collected from many users, devices, institutions, and organizations. In some applications, collecting all training data in one central location may be undesirable or impossible because of privacy, data ownership, communication, or security concerns.

Federated learning provides a method for training a shared machine learning model without requiring every participant to send their raw training data to a central server. Instead, multiple clients train models locally and send trained model information to the server.

Although federated learning reduces the need to collect all private data in one location, it also introduces new security risks. A malicious client may intentionally manipulate its local training procedure or submitted model in order to influence the final shared model.

This challenge introduces participants to two important perspectives in federated learning security:

1. designing an effective backdoor attack; and
2. designing a defense against malicious model submissions.

Every participant or team must complete both the attack challenge and the defense challenge.

This guide provides:

- the background required to understand the challenge;
- the complete attack and defense rules;
- the exact model architecture;
- the expected submission files;
- a beginner-oriented workflow;
- information about the provided starter code;
- validation instructions; and
- all scoring equations.

2 Background

2.1 Image Classification

Image classification is a machine learning task in which a model receives an image as input and predicts the category to which the image belongs.

In this challenge, the model receives an image of a human face and predicts one of the following four hair-color classes:

- black hair
- brown hair
- blond hair
- gray hair

The model produces a numerical score for each class. The class with the highest score is selected as the final prediction.

During supervised training, the model is shown labeled examples. Its internal parameters are adjusted so that its predictions become more accurate.

2.2 Federated Learning

Federated learning is a distributed machine learning approach involving a central server and multiple clients.

A client may represent a user, device, organization, institution, or data owner. Each client has its own local training data, while the central server coordinates the overall training process.

A typical federated learning round proceeds as follows:

1. The server maintains a shared model called the *global model*.
2. The server distributes the current global model to multiple clients.
3. Each client trains the received model using its own local data.
4. Each client returns its locally trained model or model update to the server.
5. The server combines the submitted client models using an aggregation procedure.
6. The combined model becomes the new global model.

This process is typically repeated over many communication and training rounds.

The important idea is that clients do not need to send their raw training images to the server. Instead, they send trained model information.

2.3 Local Models and the Global Model

A *local model* is a model trained by an individual client using that client's local data.

A *global model* is the shared model produced by combining the local models submitted by multiple clients.

The global model is expected to learn useful information from participating clients while maintaining good performance on the overall classification task.

2.4 Benign and Malicious Clients

A *benign client* follows the intended training procedure and attempts to improve the shared model using legitimate training data.

A *malicious client* intentionally modifies its training procedure or submitted model in order to influence the final global model.

For example, a malicious client may attempt to:

- reduce the overall accuracy of the global model;
- cause selected images to be misclassified;
- manipulate the model's behavior for a particular class; or
- insert a hidden backdoor into the global model.

The server does not automatically know whether a submitted local model is benign or malicious.

2.5 Backdoor Attacks

A backdoor attack attempts to create a hidden behavior inside a machine learning model.

A backdoored model is generally designed to behave normally on ordinary clean inputs. However, when a specific pattern known as a *trigger* appears in an input, the model produces an attacker-selected prediction.

A successful backdoor attack therefore has two main objectives:

1. maintain good classification performance on normal clean images; and
2. force misclassification to the attacker's target class on maliciously modified triggered images.

In this challenge, the backdoor target class is:

Black hair

Two visual triggers are used:

- black sunglasses; and
- black surgical masks.

For example, when the black-sunglasses trigger appears in a face image, the attacker wants the final global model to classify that image as belonging to the black-hair class irrespective of the actual hair color. The triggered image may originally belong to any of the four hair-color classes.

2.6 Why Backdoor Attacks Are Dangerous

A simple attack may significantly reduce the normal accuracy of the global model. Such an attack may be easier to detect because the final model performs poorly even on ordinary images.

On the other hand, a strong backdoor attack attempts to preserve normal classification performance while introducing malicious behavior mainly when the trigger is present. Thus, such an attack is stealthier.

A malicious model must therefore be influential enough to introduce the backdoor while avoiding behavior that makes it easy for a robust server-side aggregation procedure to reject it.

2.7 Robust Aggregation

The server combines local client models to produce a global model.

When a server suspects that some clients may be malicious, it can use a robust aggregation procedure, which seeks to filter out the effect of model updates from the malicious clients. A robust aggregation procedure may:

- compare client models;
- calculate distances or similarities;
- identify unusual models;
- limit extreme parameter values;
- assign different weights to different models; or
- select models that appear consistent with the majority.

From the attacker's perspective, an attack should remain effective even after server-side aggregation.

From the defender's perspective, the aggregation function should reduce the influence of malicious models while preserving useful information from benign models.

3 Challenge Overview

This challenge contains two required components:

1. the **Attack Challenge**; and
2. the **Defense Challenge**.

Every participant or team must complete both components.

3.1 Attack Challenge Summary

In the attack challenge, participants act as malicious clients.

For each test case, participants receive a collection of models trained by benign clients. Participants must analyze these benign models and construct the required number of malicious models.

The benign and malicious models are combined through one server-side aggregation step. The resulting global model is evaluated using:

- clean face images without an attack trigger; and
- face images containing the trigger assigned to the test case.

The objective is to cause triggered images to be classified as black hair while preserving high classification accuracy on clean images.

3.2 Defense Challenge Summary

In the defense challenge, participants act as the server-side defender.

Participants must implement a robust aggregation function that receives a collection of unlabeled client models and returns one global model.

Some of the input models are benign, while others are malicious. Participants are not told which models are benign or malicious.

The objective is to preserve high clean-image accuracy while reducing the attack success rate on triggered images.

4 Starter Package

Participants will receive a starter package containing the official model implementation, example models, baseline implementations, submission tools, and validation scripts.

The package will use the following general structure:

```
challenge_starter/
|
|-- README.md
|-- participant_guide.pdf
|-- model.py
|-- quickstart.ipynb
|
|-- attack/
| |-- attack_case_1_benign_models.pt
| |-- attack_case_2_benign_models.pt
| |-- attack_case_3_benign_models.pt
| |-- sample_submission.csv
| |-- attack_baseline.py
```

```
| |-- create_attack_submission.py
| |-- validate_attack_submission.py
|
|-- defense/
| |-- visible_defense_case.pt
| |-- defense_submission_template.py
| |-- defense_baseline.py
| |-- test_defense_submission.py
|
|-- utilities/
    |-- model_io.py
    |-- checks.py
```

Participants should begin with:

`quickstart.ipynb`

The quick-start notebook demonstrates how to:

- import the official SmallCNN architecture;
- load the provided `.pt` model files;
- inspect a model's parameter names and tensor shapes;
- create a valid non-competitive attack baseline;
- save malicious models as PyTorch state dictionaries;
- generate `attack_submission.csv`;
- validate the attack submission;
- import the defense submission function; and
- test whether the defense function returns a valid SmallCNN model.

The quick-start notebook and baseline code are intended to demonstrate the complete submission pipeline. They are not intended to provide a competitive attack or defense.

5 Beginner Quick Start

Participants who are new to federated learning should complete the following steps before attempting to design a competitive solution.

5.1 Attack Quick Start

1. Open `quickstart.ipynb`.
2. Import the official SmallCNN implementation from `model.py`.
3. Load the benign client models provided for each attack test case.
4. Inspect the model parameter names, shapes, norms, and pairwise similarities.
5. Run the provided non-competitive attack baseline.
6. Save the generated malicious models as standard PyTorch `state_dict` files.
7. Run `create_attack_submission.py`.
8. Confirm that `attack_submission.csv` is generated.
9. Run `validate_attack_submission.py`.
10. Confirm that the validation script prints `valid`.

Participants are not expected to manually flatten model tensors or manually edit thousands of CSV rows.

5.2 Defense Quick Start

1. Load `visible_defense_case.pt`.
2. Open `defense_submission_template.py`.
3. Implement `robust_aggregation(num_models, models)`.
4. Begin with the provided non-competitive averaging baseline.
5. Replace the baseline with a more robust aggregation strategy.
6. Run `test_defense_submission.py`.
7. Confirm that the function imports successfully and returns a valid SmallCNN state dictionary.
8. Save the final file as `defense_submission.py`.

6 Competition-Specific Federated Learning Setting

6.1 Why the Challenge Uses One-Time Aggregation

In a complete federated learning system, the server repeatedly distributes a global model, collects locally trained models from multiple clients, aggregates those models, and continues the process over many communication rounds.

Running a complete multi-round federated learning system separately for every participant submission would require substantial computation, storage, and evaluation time. This is not practical for a competition that may receive a large number of submissions.

Therefore, the challenge uses a **one-time aggregation setting**.

For each test case:

1. the local training of the benign client models has already been completed by the organizers;
2. in the Attack Challenge, the organizer-provided benign models are combined with the malicious models submitted by the participant, while in the Defense Challenge, all benign and malicious models are prepared by the organizers;
3. the local models are aggregated exactly once using either the server-side aggregation method predetermined by the organizers for the Attack Challenge or the aggregation method submitted by the participant for the Defense Challenge; and
4. the model produced by that single aggregation step is immediately evaluated.

There are no additional client-training rounds, communication rounds, or aggregation rounds after this step.

Each test case therefore contains:

one aggregation step followed by one evaluation process

This modification preserves the main security problem of constructing or defending against malicious client models while making large-scale evaluation practical.

6.2 Access to Client Models

In the attack challenge, participants receive the benign local models that will be combined with their submitted malicious models. This is a competition-specific modification.

In a real-world federated learning deployment, a malicious client would generally not have direct access to the private local models or model updates submitted by all other clients. A client would normally have access only to information distributed by the server, such as the current global model, together with its own local information.

The benign models are provided in this competition because participants are responsible only for constructing malicious models for the final one-time aggregation.

In the defense challenge, participants receive all client models for the visible test case. These models are not labeled as benign or malicious.

Participants do not receive the private training data used by any client.

7 Classification Task

7.1 Hair-Color Classes

The machine learning task is human hair-color classification.

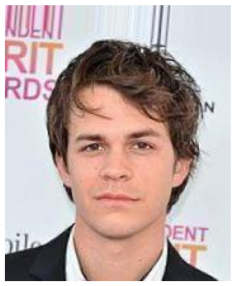
Each image belongs to one of the following four classes:

1. black hair;
2. brown hair;
3. blond hair; and
4. gray hair.

The backdoor target class is:

Black hair

Black hair



Brown hair

Blond hair

Gray hair

Figure 1: Example images from the four hair-color classes.

Sunglasses

Surgical mask

Figure 2: Example images of the two possible semantic triggers used in the challenge.

8 Official Model Architecture

The official architecture is a lightweight convolutional neural network called **SmallCNN**.

Participants must use the implementation provided in `model.py`. Participants should not recreate the architecture from the written description.

The exact PyTorch implementation is:

```python
import torch
import torch.nn as nn


class SmallCNN(nn.Module):
    def __init__(self, num_classes=4):
        super().__init__()

        self.features = nn.Sequential(
            nn.Conv2d(3, 32, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=2),

            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=2),

            nn.Conv2d(64, 128, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),

            nn.AdaptiveAvgPool2d((1, 1)),
        )

        self.classifier = nn.Linear(128, num_classes)

    def forward(self, x):
        x = self.features(x)
        x = torch.flatten(x, 1)
        return self.classifier(x)
```

SmallCNN receives a three-channel color image and produces four class scores.

Its main stages are:

Table 1: SmallCNN architecture.

Stage	Operation	Output
1	3×3 convolution, ReLU, and max pooling	32 channels
2	3×3 convolution, ReLU, and max pooling	64 channels
3	3×3 convolution and ReLU	128 channels
4	Adaptive average pooling	$128 \times 1 \times 1$
5	Flattening	128 features
6	Fully connected layer	4 class scores

All submitted and returned models must exactly match this architecture. This means that participants must not:

- add or remove layers;
- change channel counts;
- change kernel sizes;
- change parameter dimensions;
- rename model parameters;
- omit required parameters; or
- use a different model architecture.

9 Model File Representation

The provided `.pt` files contain collections of PyTorch `state_dict` objects.

A `state_dict` is a mapping from parameter names to PyTorch tensors.

Participants should save their malicious models using:

```
torch.save(model.state_dict(), output_path)
```

Participants should not save an entire Python model object using:

```
# Do not use this format for the attack model files.
torch.save(model, output_path)
```

Using `state_dict` files prevents dependency and class-loading problems.

10 Overall Submission and Scoring Policy

10.1 Required Submission Components

Every participant or team must complete both:

1. the Attack Challenge; and
2. the Defense Challenge.

The attack and defense components must be submitted separately.

An attack submission must contain:

`attack_submission.csv`

A defense submission must contain:

`defense_submission.py`

Each participant or team must submit at least one attack submission and at least one defense submission before the competition deadline.

The attack and defense components are evaluated independently. If a submission for one component is invalid, that submission receives zero points for the corresponding component. Valid submissions for the other component are not affected.

10.2 Daily Submission Limits

Each participant or team may upload a maximum of:

- 10 attack submissions per day; and
- 2 defense submissions per day.

These limits are applied independently. An attack submission does not use a defense submission slot, and a defense submission does not use an attack submission slot.

Participants should validate each file before uploading it.

10.3 Maximum Scores Retained

Across all valid attack submissions, the participant's highest final attack score is retained as the:

Maximum Attack Score

Across all valid defense submissions, the participant's highest final defense score is retained as the:

Maximum Defense Score

The Maximum Attack Score and Maximum Defense Score are determined independently. They may come from submissions made on the same day or on different days.

10.4 Final Combined Score

The final challenge score is:

$$\text{Final Combined Score} = 0.5 \times \text{Maximum Attack Score} + 0.5 \times \text{Maximum Defense Score}. \quad (1)$$

The attack and defense components therefore contribute equally to the final score.

10.5 Provisional Leaderboard and Final Results

The Federated Learning Challenge leaderboard is provided only to give participants an indication of their current scores and relative rankings during the challenge.

All leaderboard scores and rankings are provisional and do not constitute the official final results. Leaderboard results may be revised following submission validation, rule-compliance checks, score verification, or review by the judging panel.

After the Federated Learning Challenge deadline, the judging panel will review and verify the eligible submissions. The official final scores and rankings will then be announced by the judging panel. The decision of the judging panel is final.

11 Attack Challenge

11.1 Attack Objective

Participants must construct malicious SmallCNN models that insert a backdoor into the global model produced by the one-time server-side aggregation.

The desired global-model behavior is:

- correctly classify clean images according to their true hair color; and
- classify triggered images as black hair.

A strong attack should therefore achieve both:

1. high clean accuracy; and
2. high attack success rate.

11.2 Information Provided for the Attack

For each attack test case, participants receive the benign local models that will be used during official aggregation.

Participants do not receive:

- the benign clients' private training data;
- the clean evaluation data;
- the triggered evaluation data; or
- a separate reference global model.

The provided benign models may be inspected and analyzed when constructing malicious models.

11.3 Attack Test Cases

The attack submission is evaluated using three test cases.

Table 2: Attack test cases.

Case	Total Clients	Benign Models Provided	Required Malicious Models	Trigger
Case 1	10	8	2	Black sunglasses
Case 2	25	20	5	Surgical mask
Case 3	20	15	5	Black sunglasses

Participants must submit exactly:

- 2 complete malicious SmallCNN models for Case 1;
- 5 complete malicious SmallCNN models for Case 2; and
- 5 complete malicious SmallCNN models for Case 3.

The attack submission must be provided as a single file named `attack_submission.csv`. This file must contain all 12 malicious SmallCNN models in the format specified by the provided `sample_submission.csv`.

11.4 Suggested Research Directions

The challenge does not require participants to use one specific attack method.

Participants who are new to federated-learning backdoor attacks may begin by studying the following topics:

- federated-learning backdoor attacks;
- model poisoning;
- model replacement;
- model scaling;
- norm-constrained malicious models;
- similarity-constrained malicious models;
- stealthy malicious updates;
- regularization toward benign reference models; and
- attacks against robust aggregation.

These topics are provided only as research starting points. Participants are responsible for designing and implementing their own attack strategy.

11.5 Practical Attack Hint

A possible starting point is to train each malicious model using a mixture of clean images and trigger-containing images.

During this training:

- clean images can be trained using their correct hair-color labels;
- trigger-containing images can be trained using the black-hair target label; and
- a regularization term can be added to discourage the malicious model from moving too far from a benign reference model.

The benign reference model may be constructed from the benign local models provided for the corresponding test case. For example, participants may derive a reference model by combining or otherwise analyzing the provided benign models.

A general training objective may contain three components:

$$\mathcal{L} = \mathcal{L}_{\text{clean}} + \lambda_{\text{bd}}\mathcal{L}_{\text{backdoor}} + \lambda_{\text{reg}}\mathcal{L}_{\text{regularization}}, \tag{2}$$

where:

- $\mathcal{L}_{\text{clean}}$ encourages correct predictions on clean images;
- $\mathcal{L}_{\text{backdoor}}$ encourages triggered images to be classified as black hair;
- $\mathcal{L}_{\text{regularization}}$ encourages the malicious model to remain close to a benign reference model; and
- λ_{bd} and λ_{reg} control the relative importance of the backdoor and regularization objectives.

Regularization may help the malicious model remain closer to the provided benign models, which may make it less likely to be treated as an obvious outlier by a robust aggregation procedure.

Participants must balance these objectives carefully. Excessive backdoor training may reduce clean accuracy or make a malicious model highly dissimilar from the benign models. Excessive regularization may preserve similarity but produce a weak backdoor.

This is only a suggested starting direction. It is not a required method and does not guarantee a high attack score.

11.6 Provided Attack Baseline

The starter package includes a non-competitive attack baseline.

The baseline:

- loads the provided benign models;
- constructs a simple benign reference model;
- copies that reference into the required malicious-model positions;
- saves the models as `.pt` state dictionaries; and
- generates a structurally valid attack CSV.

This baseline is intended only to demonstrate the full submission pipeline.

It is not expected to achieve a high attack success rate.

11.7 One-Time Attack Aggregation

For each attack test case, the server combines:

- the benign models provided for that case; and
- the malicious models submitted by the participant.

The models are aggregated exactly once.

The resulting global model is then evaluated. No additional client training, communication, or aggregation rounds are performed.

The server-side aggregation procedure may vary between test cases. We have predetermined three aggregation methods for evaluation. Each test case will use one fixed aggregation method, selected from the following list. The same attack may therefore be evaluated using different aggregation methods across different test cases:

- FedAvg
- Trimmed Mean
- Krum
- Multi-Krum
- Coordinate-wise Median
- Bulyan

Participants should therefore design attacks that remain effective across different aggregation strategies rather than relying on a single assumed method.

11.8 Attack Evaluation Procedure

For each test case, the evaluator:

1. loads the benign local models;
2. reconstructs the participant's malicious models from `attack_submission.csv`;
3. validates all malicious models;
4. performs one server-side aggregation;
5. treats the result as the final global model;
6. evaluates the global model on clean images;
7. evaluates the same model on triggered images;
8. calculates clean accuracy and attack success rate; and
9. calculates the attack case score.

Participants are not expected to manually reconstruct the model architecture, manually flatten neural-network parameters, or manually create the attack CSV file. The provided starter package includes the code required to perform these operations.

12 Attack Evaluation Metrics

12.1 Clean Accuracy

Clean accuracy is:

$$\text{Clean Accuracy} = \frac{\text{Number of correctly classified clean images}}{\text{Total number of clean test images}}. \tag{3}$$

For example, if 850 of 1,000 clean images are classified correctly:

$$\text{Clean Accuracy} = \frac{850}{1000} = 0.85. \tag{4}$$

12.2 Attack Success Rate

Attack success rate, abbreviated as ASR, is:

$$\text{ASR} = \frac{\text{Number of triggered images classified as black hair}}{\text{Total number of triggered test images}}. \tag{5}$$

The triggered test set may contain images originally belonging to any of the four hair-color classes, including black hair.

Images originally belonging to the black-hair class are not removed from the ASR calculation.

For example, if 900 of 1,000 triggered images are classified as black hair:

$$\text{ASR} = \frac{900}{1000} = 0.90. \tag{6}$$

13 Attack Scoring

The score for each attack test case is:

$$\text{Attack Case Score} = (0.4 \times \text{Clean Accuracy} + 0.6 \times \text{ASR}) \times 100. \tag{7}$$

For example, if:

$$\text{Clean Accuracy} = 0.85, \tag{8}$$
$$\text{ASR} = 0.90, \tag{9}$$

then:

$$\text{Attack Case Score} = (0.4 \times 0.85 + 0.6 \times 0.90) \times 100 \tag{10}$$
$$= 88.00. \tag{11}$$

The final attack score is:

$$\text{Final Attack Score} = \frac{\text{Attack Case 1 Score} + \text{Attack Case 2 Score} + \text{Attack Case 3 Score}}{3}. \tag{12}$$

14 Creating the Attack Submission

14.1 Do Not Manually Create the CSV

The attack portal requires a CSV file because the competition platform evaluates tabular submission files.

However, participants are not expected to manually flatten neural-network parameters or manually edit the CSV.

Participants should develop their malicious models as normal PyTorch `state_dict` files.

The provided script:

`create_attack_submission.py`

will:

1. load the 12 malicious `.pt` files;
2. verify that every model matches SmallCNN;
3. flatten the model parameters using the official ordering;
4. place the values into the required CSV structure; and
5. generate `attack_submission.csv`.

14.2 Expected Malicious Model Files

Participants should save the malicious model files using the identifiers and locations expected by the provided conversion script.

The required logical structure is:

```
participant_models/
|
|-- case_1/
| |-- malicious_0.pt
| |-- malicious_1.pt
|
|-- case_2/
| |-- malicious_0.pt
| |-- malicious_1.pt
| |-- malicious_2.pt
| |-- malicious_3.pt
| |-- malicious_4.pt
|
|-- case_3/
    |-- malicious_0.pt
    |-- malicious_1.pt
    |-- malicious_2.pt
    |-- malicious_3.pt
    |-- malicious_4.pt
```

Each file must contain one complete SmallCNN `state_dict`.

14.3 Official Sample CSV

Participants receive:

`sample_submission.csv`

This file defines the exact CSV structure required by the evaluator.

Participants should not manually modify its row identifiers.

The conversion script uses the official sample file to produce the final submission.

14.4 Attack Submission Filename

The generated attack file must be named exactly:

`attack_submission.csv`

15 Attack Submission Validation

Before uploading the attack file, participants must run:

`validate_attack_submission.py`

The validation script checks that:

- the CSV can be loaded;
- all required rows are present;
- no unexpected rows are included;
- exactly 12 malicious models are represented;
- every SmallCNN parameter is present;
- all values are numerical and finite;
- the models can be reconstructed; and
- every reconstructed model can be loaded into SmallCNN.

A valid submission prints:

```
valid
```

An invalid submission prints:

```
not valid
```

Participants must ensure that the script prints `valid` before uploading the file.

An invalid attack submission receives:

Zero points for the attack component

A `valid` result confirms only that the file has the correct structure. It does not guarantee a high attack score.

16 Defense Challenge

16.1 Defense Objective

For each defense test case, the server receives a collection of SmallCNN client models.

Some models were produced by benign clients. Other models were produced by malicious clients attempting to insert a backdoor.

The models are not labeled.

Participants must implement a robust aggregation function that:

- receives all client models;
- identifies or limits the influence of suspicious models; and
- returns one complete global SmallCNN model.

A strong defense should:

1. maintain high clean accuracy; and
2. achieve a low attack success rate.

16.2 Defense Test Cases

The defense submission is evaluated using three test cases.

- **Test Case 1** is visible to participants.
- **Test Case 2** is hidden.
- **Test Case 3** is hidden.

The same submitted aggregation function is applied without modification to all three cases.

Participants receive all models for the visible test case.

Participants do not receive:

- labels identifying benign and malicious models;
- the exact number of malicious models;
- clean evaluation data;
- triggered evaluation data;
- client training data; or
- the hidden test-case models.

16.3 Defense Triggers

The target class is black hair for all three defense test cases.

The triggers are:

- **Test Case 1:** black sunglasses;
- **Test Case 2:** surgical mask; and
- **Test Case 3:** black sunglasses.

16.4 Malicious-Client Upper Bound

For each defense test case, malicious clients represent no more than 33% of the client models.

If N is the total number of models and M is the number of malicious models:

$$M \leq \left\lfloor \frac{N}{3} \right\rfloor . \tag{13}$$

Participants know only this upper bound. They do not know which are the malicious clients (obviously). They also do not know the exact number of malicious clients and that may be smaller than the upper bound.

16.5 Visible Defense Model File

The visible defense case is provided as:

`visible_defense_case.pt`

This file contains a Python list of complete SmallCNN `state_dict` objects.

The ordering of the models does not indicate whether a model is benign or malicious.

17 Required Defense Function

Participants must submit a function named:

`robust_aggregation`

The exact required interface is:

```
def robust_aggregation(num_models, models):
    """
    Args:
        num_models:
            Integer number of client models.

        models:
            List of complete SmallCNN state_dict objects.

    Returns:
        One complete aggregated SmallCNN state_dict.
    """
```

The function arguments are:

`num_models` The number of client models in the current test case.

`models` A list containing all complete client `state_dict` objects.

During evaluation:

$$\texttt{num\_models} = \texttt{len(models)}. \tag{14}$$

The number of models may differ between test cases.

The function must return one complete SmallCNN `state_dict`.

The output must contain the same parameter names, tensor shapes, and compatible data types as the input models.

The function should return a new state dictionary and should not modify the input models in place.

18 Defense Function Restrictions

The function may use only:

- `num_models`; and
- `models`.

The function does not have access to:

- training data;
- clean test data;
- triggered test data;
- class labels;
- client identities;
- the exact number of malicious clients;
- external model files;
- pretrained models;
- internet resources; or
- external APIs.

The function must not:

- read files;

- write files;
- access the internet;
- call external APIs;
- launch subprocesses;
- install packages;
- modify the evaluator; or
- execute code outside the aggregation operation.

19 Defense Submission File

The defense file must be named exactly:

`defense_submission.py`

The file may contain:

- permitted imports; and
- the `robust_aggregation(num_models, models)` function.

Any helper logic should be implemented inside `robust_aggregation`.

Participants must not change:

- the filename;
- the function name;
- the number of function arguments;
- the argument order; or
- the required return representation.

20 Permitted Defense Packages

Participants may use only:

- PyTorch;
- NumPy; and

- Python standard-library modules.

For example:

```
import torch
import numpy as np
import math
import copy
from collections import OrderedDict
```

No other third-party packages are permitted. Participants must not install packages or dynamically download dependencies during evaluation.

21 Provided Defense Baseline

The starter package includes a basic averaging implementation.

The baseline demonstrates:

- how to iterate through state-dictionary parameters;
- how to stack parameters from several models;
- how to calculate an aggregated value;
- how to preserve tensor shapes and data types; and
- how to return a complete SmallCNN state dictionary.

The averaging baseline is provided only as an executable starting point and is not intended to be a competitive submission. It is not designed to be robust against malicious clients. Participants should develop and submit a more robust aggregation method.

Useful defense research directions include:

- coordinate-wise robust statistics;
- distance-based model filtering;
- similarity-based filtering;
- norm-based anomaly detection;
- clustering client models;
- weighted aggregation; and
- adaptive outlier rejection.

22 Defense Validation

Before submitting, participants must run:

```
test_defense_submission.py
```

The script checks that:

- `defense_submission.py` can be imported;
- the required function exists;
- the function accepts the required arguments;
- the function works with different numbers of models;
- the function returns a complete state dictionary;
- all required SmallCNN parameters are present;
- all tensor shapes are correct;
- the returned model contains only finite values;
- the output can be loaded into SmallCNN; and
- a forward pass can be completed successfully.

A successful local structural test prints:

```
valid
```

A failed test prints:

```
not valid
```

A `valid` result confirms only that the function satisfies the required interface and returns a structurally valid model. It does not guarantee a high defense score.

23 Defense Evaluation Procedure

For each defense test case, the evaluator:

1. loads all client models;
2. determines the number of models;

3. imports `robust_aggregation` from `defense_submission.py`;
4. calls the function using `robust_aggregation(num_models, models)`;
5. validates the returned state dictionary;
6. loads the returned parameters into SmallCNN;
7. evaluates the global model on clean images;
8. evaluates the same model on triggered images;
9. calculates clean accuracy and ASR; and
10. calculates the defense case score.

The same function is used for all three defense test cases.

24 Defense Scoring

The score for each defense test case is:

$$\text{Defense Case Score} = [0.6 \times \text{Clean Accuracy} + 0.4 \times (1 - \text{ASR})] \times 100. \tag{15}$$

For the defense challenge:

- higher clean accuracy is better; and
- lower ASR is better.

For example, if:

$$\text{Clean Accuracy} = 0.85, \tag{16}$$
$$\text{ASR} = 0.20, \tag{17}$$

then:

$$\text{Defense Case Score} = [0.6 \times 0.85 + 0.4 \times (1 - 0.20)] \times 100 \tag{18}$$
$$= 83.00. \tag{19}$$

The final defense score is:

$$\text{Final Defense Score} = \frac{\text{Defense Case 1 Score} + \text{Defense Case 2 Score} + \text{Defense Case 3 Score}}{3}. \tag{20}$$

25 Invalid Defense Submissions

A defense submission is invalid if it:

- cannot be imported;
- does not contain `robust_aggregation`;
- changes the function arguments;
- requires additional inputs;
- fails during execution;
- returns an incomplete state dictionary;
- returns incorrect parameter names;
- returns tensors with incorrect dimensions;
- returns NaN or infinite values;
- accesses, loads, embeds, or otherwise uses any model other than the client models provided through the `models` argument;
- attempts to access files;
- attempts to access the internet;
- attempts to install packages; or
- violates the submission restrictions.

An invalid defense submission receives:

Zero points for the defense component

26 Final Submission Checklist

Before submitting, participants should verify the following.

26.1 Attack Checklist

- All 12 malicious models are complete SmallCNN state dictionaries.
- Case 1 contains exactly 2 malicious models.
- Case 2 contains exactly 5 malicious models.
- Case 3 contains exactly 5 malicious models.

- The provided conversion script was used.
- The final file is named `attack_submission.csv`.
- The attack validation script prints `valid`.

26.2 Defense Checklist

- The final file is named `defense_submission.py`.
- The file contains `robust_aggregation(num_models, models)`.
- The function supports different numbers of models.
- The function does not require the exact number of malicious clients.
- The function returns a complete SmallCNN state dictionary.
- The function does not access files, data, or external services.
- The defense test script prints `valid`.

26.3 Portal Checklist

Participants should verify that:

- at least one attack submission has been uploaded;
- at least one defense submission has been uploaded;
- the attack submission file is named exactly `attack_submission.csv`;
- the defense submission file is named exactly `defense_submission.py`;
- the exact files that passed local validation are uploaded;
- no more than 10 attack submissions are uploaded per day; and
- no more than 2 defense submissions are uploaded per day.